📑 جزوه سیستم عامل - فصل ۶: ابزار همگامی (بخش سوم)

(ابزارهای نرم‌افزاری سطح بالا: Mutex و Semaphore)

۷. قفل‌های انحصار متقابل (Mutex Locks)

چون استفاده از دستورات سخت‌افزاری (مثل CAS) برای برنامه‌نویسان دشوار است، طراحان سیستم‌عامل ابزارهای سطح بالاتری ارائه داده‌اند. ساده‌ترین آن‌ها **Mutex** است.

- **ریشه نام:** Mutex مخفف **Mut**ual **Ex**clusion (انحصار متقابل) است.
- **مکانیزم:**
  1. پردازه قبل از ورود به بخش بحرانی، قفل را **می‌گیرد (acquire)**.
  2. کارش را انجام می‌دهد.
  3. قفل را **آزاد می‌کند (release)**.

توابع اصلی Mutex

یک متغیر available داریم (True یعنی قفل آزاد است).

1. **تابع** `acquire()`:

```
acquire() {
    while (!available); // انتظار مشغول (Busy Wait): تا وقتی قفل نیست، همین‌جا بچرخ
    available = false;  // قفل را گرفتم (دیگر آزاد نیست)
}
```

2. **تابع** `release()`:

```
release() {
    available = true;   // قفل را آزاد کردم
}
```

نکته: فراخوانی این توابع باید به صورت **اتمیک** باشد (معمولاً با همان دستورات سخت‌افزاری CAS پیاده‌سازی می‌شوند).

مفهوم "قفل چرخشی" (Spinlock)

در پیاده‌سازی بالا، پردازه در حلقه while مدام می‌چرخد تا قفل آزاد شود. به این نوع قفل، **Spinlock** می‌گویند.

- **عیب:** پردازنده را مشغول نگه می‌دارد و انرژی مصرف می‌کند.
- **مزیت:** **Context Switch** ندارد. (تغییر متن پردازه زمان‌بر است).
- **کاربرد:** اگر زمان انتظار برای قفل **کوتاه** باشد، Spinlock بهتر از این است که پردازه را به خواب ببریم و دوباره بیدار کنیم (چون هزینه خواب/بیدار بیشتر از چند لحظه چرخیدن است). در سیستم‌های چند هسته‌ای بسیار پرکاربرد است.

۸. سمافورها (Semaphores)

سمافور ابزاری قدرتمندتر از Mutex است که شبیه چراغ راهنمایی عمل می‌کند.

- توسط **دایکسترا (Dijkstra)** معرفی شد.
- یک متغیر صحیح (int S) است که فقط با دو عملیات اتمیک قابل دسترسی است: wait() و signal().

عملیات اصلی سمافور:

عملیات wait(S) (یا P):

- مقدار سمافور را چک می‌کند.
- اگر مثبت بود، یکی کم می‌کند و ادامه می‌دهد.
- اگر صفر (یا منفی) بود، منتظر می‌ماند.

```
} wait(S)
while (S <= 0); // انتظار
--S;
{
```

. **عملیات** `signal(S)` (یا `V`):

- مقدار سمافور را یکی زیاد می‌کند (اعلام می‌کند منبع آزاد شد).

```
} signal(S)
++S;
{
```

انواع سمافور:

سمافور شمارشی (Counting Semaphore):

- دامنه مقدارش محدود نیست.
- **کاربرد:** کنترل دسترسی به **تعداد محدودی منبع** (مثلاً ۳ پرینتر داریم، سمافور را روی ۳ می‌گذاریم).

سمافور دودویی (Binary Semaphore):

- مقدارش فقط ۰ یا ۱ است.
- **کاربرد:** دقیقاً مثل قفل Mutex عمل می‌کند.

مثال کاربردی: هماهنگی (Synchronization) با سمافور

فرض کنید می‌خواهیم پردازه 2P دستور 2S را حتماً **بعد از** اینکه پردازه 1P دستور 1S را اجرا کرد، اجرا کند.

سمافور مشترک synch = 0

کد پردازه 1P:

```
;S1            // اول این اجرا می‌شود
;signal(synch)  // حالا اجازه می‌دهم (synch می‌شود ۱)
```

کد پردازه P2:

```
;wait(synch)    // اینجا گیر می‌کند چون synch صفر است. وقتی P1 سیگنال داد، رد می‌شود.
;S2             // حالا اجرا می‌شود
```

۹. پیاده‌سازی سمافور بدون "انتظار مشغول" (Busy Wait)

مشکل تعریف قبلی سمافور این بود که در حلقه `while` می‌چرخید (Spinlock). برای حل این مشکل، پردازه را به **خواب (Sleep)** می‌بریم.

ساختار جدید سمافور:

```
} typedef struct
  ;int value
  ;struct process *list // لیستی از پردازه‌های منتظر (صف انتظار)
;semaphore {
```

عملیات جدید wait و signal:

:(wait(S

- مقدار را کم می‌کند (--S->value).
- اگر منفی شد (یعنی منبعی نیست)، پردازه را به لیست list اضافه می‌کند و آن را **خواب می‌کند** (block).

:(signal(S

- مقدار را زیاد می‌کند (++S->value).
- اگر مقدار هنوز غیرمثبت بود (یعنی کسی در صف خوابیده)، یک پردازه را از لیست برمی‌دارد و **بیدار می‌کند** (wakeup).

- **نکته کلیدی:** در این روش، مقدار سمافور می‌تواند **منفی** شود. قدر مطلق عدد منفی، نشان‌دهنده **تعداد پردازه‌های منتظر در صف** است. (مثلاً S = -3 یعنی ۳ نفر در صف هستند).

چالش پیاده‌سازی در چند هسته‌ای (SMP)

عملیات wait و signal باید اتمیک باشند.

- در تک‌پردازنده: با غیرفعال کردن وقفه‌ها (Interrupts) حل می‌شود.

- در چندپردازنده: باید از تکنیک‌های سخت‌افزاری مثل compare_and_swap یا Spinlock استفاده کنیم تا از ساختار خودِ سمافور محافظت کنیم.

(پایان بخش سوم - صفحات ۳۷۲ تا ۳۷۷ کتاب)